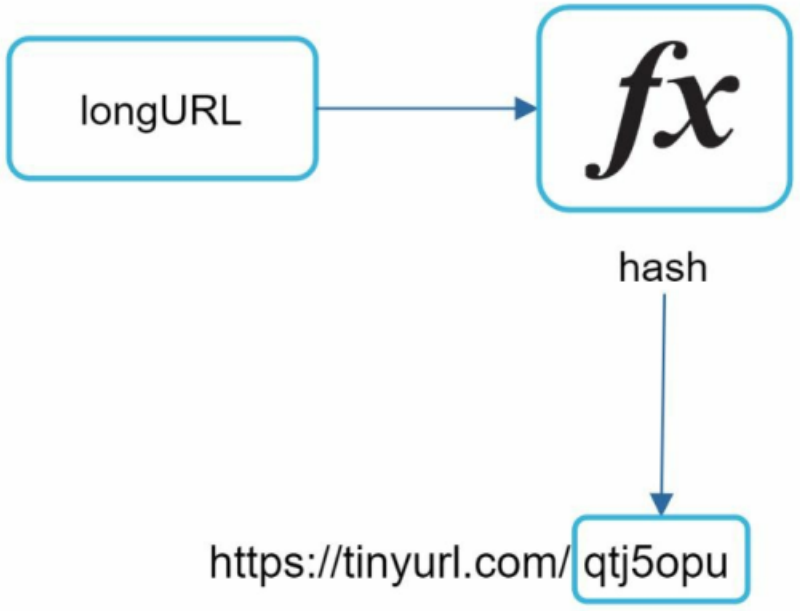
Design a URL Shortener - URL Shortening

1. Overview

- Short URL format:

```
www.tinyurl.com/{hashValue}
```

- Goal:
    - Convert **long URL → short URL**



2. Core Idea

- Use a **hash function f(x)**

```
None
f(longURL) → hashValue
```

- Final short URL:

```
None
shortURL = domain + hashValue
```

3. Example

```
None
Input:

https://www.systeminterview.com/q=chat...


Output:

https://tinyurl.com/y7keocwj
```

4. Requirements of Hash Function

1. Uniqueness

- Each **longURL → unique hashValue**

Avoid collisions:

- Two different URLs should not produce same hash

2. Reversibility (Mapping)

- Each **hashValue → original longURL**

Important:

- We do NOT reverse mathematically
- We **store mapping in database**

5. Basic Flow

Step-by-step

```
None
1. Client sends long URL

2. System generates hashValue using f(x)

3. Store mapping:

   <hashValue, longURL>

4. Return short URL to client
```

6. Storage Design

- Use **key-value store**

```
None
<hashValue, longURL>
```

Example:

```
None
y7keocwj → https://www.systeminterview.com/...
```

7. Key Challenges

Collision Handling

- Two URLs may generate same hash
- Must detect and resolve

Hash Length

- Shorter hash → better UX
- But:
    - Too short → higher collision risk

Scalability

- Must handle:
    - Millions of URL generations daily

8. Key Takeaway

URL shortening depends on:

- Efficient **hash generation**
- Reliable **mapping storage**
- Proper **collision handling**

Detailed hash function design → covered in deep dive